

R5.07

Automatisation de la chaîne de production

Jean-Yves Colin
IUT du Havre
Département Informatique
v1.03- 2023-2025

Le ‘workflow’ de GitHub



Workflow : GitHub



- GitHub
 - est un service web,
 - qui vise à être une « plate-forme » d'hébergement et de gestion de développement de logiciels,
 - et est basé sur l'outil de gestion de versions Git.

Workflow : GitHub (suite)



- Quelques définitions :
 - '*repository*' ("dépôt") : zone de stockage d'un projet.
 - Elle contient des fichiers et des répertoires
 - Elle gère au cours du temps les différentes versions de ces fichiers et répertoires (à l'aide de Git).
 - Il peut contenir aussi des fichiers de données, des images, des vidéos etc.
 - '*branch*' ("branche") : une version parallèle d'un "dépôt" ou d'une autre "branche".

Workflow : GitHub (suite)



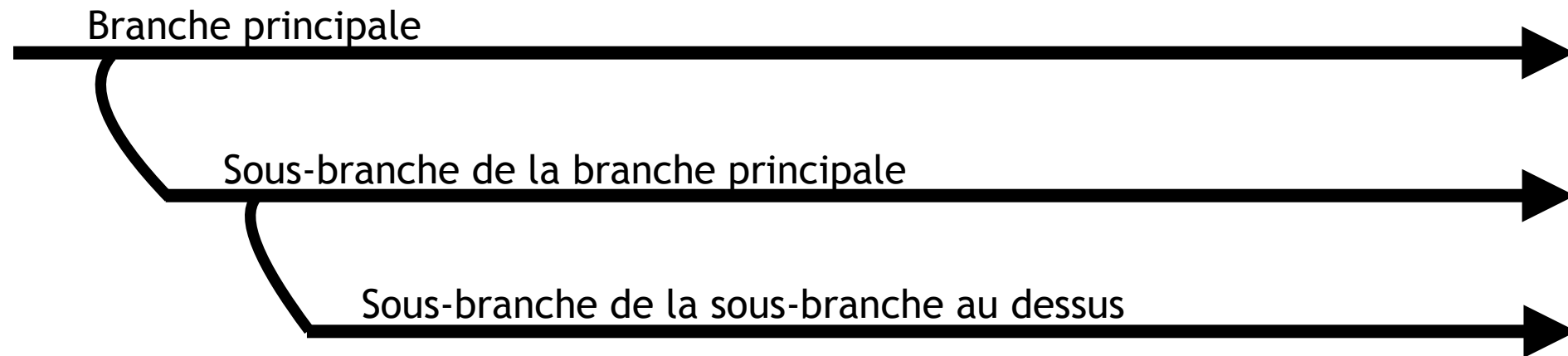
- 'main branch' ("branche principale") :
 - version "officielle" unique du projet dans ce dépôt GitHub, telle qu'elle est **vue par un utilisateur extérieur** au projet (le reste n'est normalement pas visible de lui).
 - Créée automatiquement lors de la création du "dépôt",
 - C'est aussi la branche par défaut,
 - C'est la seule branche officiellement « permanente » dans un dépôt GitHub.

Workflow : GitHub (suite)



- *'feature branch'* ou *'topic branch'* :
 - branche supplémentaire temporaire pour l'ajout de "fonctionnalités" (ou l'insertion de corrections)
 - toujours créée par **copie** de la branche actuellement utilisée
 - qui peut être la "branche principale",
 - ou une autre "branche supplémentaire" existante.
- ...

Workflow : GitHub (suite)



Workflow : GitHub (suite)



- '*feature branch*' ou '*topic branch*' (suite) :
 - ...
 - La branche d'origine est parfois appelée branche "de base" de la branche supplémentaire.
 - Le travail fait sur cette branche n'affectera pas sa branche "de base" (tant qu'on n'a pas fait de "fusion" de ces branches).
 - ...

Workflow : GitHub (suite)



- '*feature branch*' ou '*topic branch*' (suite) :
 - ...
 - Ces branches supplémentaires sont donc là pour expérimenter et faire des corrections de bugs,
 - Il n'y a pas de risque de modifier par erreur la branche de base.
 - ...

Workflow : GitHub (suite)



- '*feature branch*' ou '*topic branch*' (suite) :
 - ...
 - Il est très fortement conseillé de créer une branche distincte par ensemble indépendant de modifications.
 - Plus tard, cela facilitera beaucoup :
 - les discussions,
 - les suivis,
 - les retours en arrière si on décide de ne garder qu'une des modifications et pas l'autre, etc.
 - ...

Workflow : GitHub (suite)

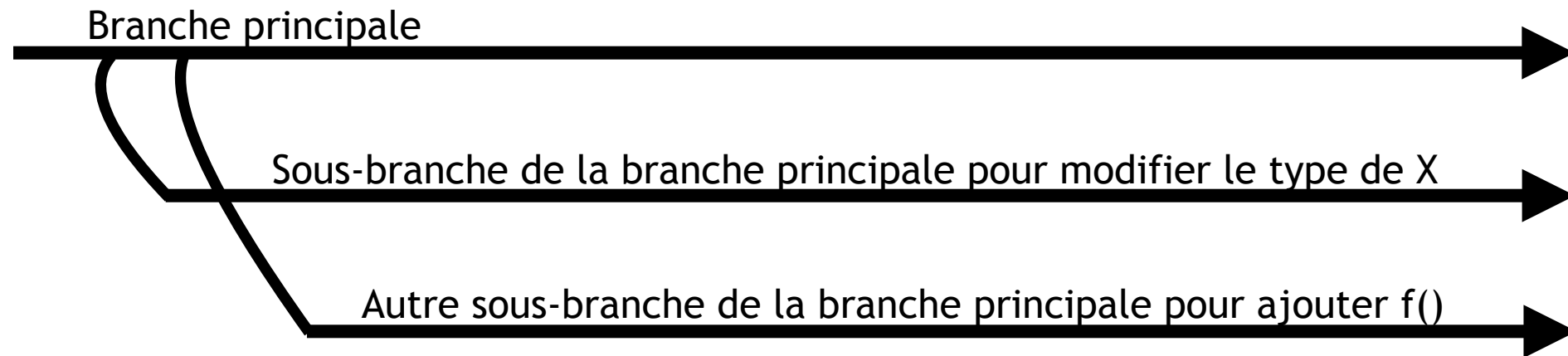


- '*feature branch*' ou '*topic branch*' (suite) :
 - ...
 - Exemple: S'il faut modifier le type d'une variable X, et aussi ajouter une fonction f() qui n'utilise pas directement cette variable, alors il vaut beaucoup mieux faire deux branches distinctes et indépendantes :
 - une branche « modifier le type de X »,
 - et une branche « ajouter fonction f() »

Workflow : GitHub (suite)



- Exemple :



Workflow : GitHub (suite)



- ‘*Commit*’ ("validation") :
 - Capture l'état à ce moment des fichiers dans cette branche. Cet « instantané » est interne à la branche,
 - Donne un identifiant unique ('hash'/'SHA') aux modifications apportées, la date des modifications et qui les a faites.
 - Il faut ajouter un petit message décrivant les modifications effectuées, pour aider à comprendre ce qui est fait comme modification dans cette branche.
- ...

Workflow : GitHub (suite)



- ‘Commit’ ("validation") (suite) :
 - ...
 - Une fois un ‘Commit’ fait, il sera possible de revenir plus tard pour cette branche « en arrière dans le temps » au moment de ce ‘Commit’,
 - par exemple en créant une nouvelle branche à partir de ce ‘Commit’ (et donc de l’état qu’il contient et qui est celui de cette branche au moment où ce ‘Commit’ a été fait), par une commande Git

Workflow : GitHub (suite)



- '*pull request*' ("demande de tirage") :
 - proposition de "fusion" ('*merge*') d'une branche vers sa branche de base.
 - Temps commun de vérification et de contrôle d'un travail effectué avant sa prise en compte officielle.
 - Toujours à faire avant une "fusion" réelle.
 - La proposition doit être décrite : qu'est-ce qu'elle apporte par rapport à sa branche de base qui existe déjà ?
 - ...

Workflow : GitHub (suite)



- '*pull request*' ("demande de tirage") (suite) :
 - ...
 - Note : mal nommé. '*merge request*' aurait été plus approprié...
 - Si elle est acceptée, les modifications dans la branche actuelle pourront être copiées dans sa branche de base ('*merge*').
 - ...

Workflow : GitHub (suite)



- '*pull request*' ("demande de tirage") (suite) :
 - Mais des vérifications sont généralement effectuées avant :
 - détection automatique des « conflits » entre versions (conflits de "concurrency");
 - généralement entre deux branches différentes qui modifient le même fichier de façons incompatibles
 - mais parfois deux branches qui suppriment des fichiers que l'autre branche utilise
 - résolution automatique quand c'est possible ('auto-merge'), sinon intervention manuelle obligatoire

Workflow : GitHub (suite)



- Exemple simple de conflit :
 - **fichier** `README.txt` **dans la branche** `Base` :
Bienvenue dans mon projet
 - **Anne le modifie dans une sous-branche** `Anne_modification` **de la**
branche `Base` :
Bienvenue dans le projet d'Anne
 - **et Bernard le modifie dans une sous-branche** `Bernard_modification`
de la branche `Base` :
Bienvenue dans le projet de Bernard

Workflow : GitHub (suite)



Ce qui se passe lors d'une demande de tirage :

1. Anne fait sa demande de tirage en premier (par exemple) pour sa sous-branche et la fusionne. La branche `main` contient maintenant dans `README.txt` :
`Bienvenue dans le projet d'Anne`
2. Ensuite, Bernard fait sa demande de tirage pour sa sous-branche. GitHub essaie de la fusionner avec la branche `main` (déjà modifiée par Anne)
3. ⚡ Problème : GitHub voit que la même ligne du fichier `README.txt` a été modifiée différemment par Anne et Bernard.
4. Résultat : Conflit de merge !

Workflow : GitHub (suite)



- GitHub va alors bloquer la fusion de Bernard et afficher :

```
<<<<<<< main
```

```
Bienvenue dans le projet d'Anne
```

```
=====
```

```
Bienvenue dans le projet de Bernard
```

```
>>>>>>> Bernard_modification
```

Workflow : GitHub (suite)



- Exemple un peu plus subtil :

- **fichier** `Toto.java` **dans la branche** `Base` :

```
double addTemp(double temp1, double temp2)
{ // les deux paramètres sont en degrés
  Celsius
  return (temp1+temp2) ;
}
```

- Un problème est constaté dans certains cas, sans trop savoir pourquoi

Workflow : GitHub (suite)



- Anne le modifie dans une sous-branche `Anne_verification` de la branche `Base`

```
double addTemp(double temp1, double temp2)
{ // les deux paramètres sont en degrés Celsius
  if (temp1 <= -273.15 || temp2 <= -273.15) {
    throw new
      IllegalArgumentException
        ("sous zéro absolu !");
  }
  return (temp1+temp2);
}
```

Workflow : GitHub (suite)



- et Bernard le modifie dans une sous-branche `Bernard_verification` de la branche `Base` pour afficher directement les paramètres :

```
double addTemp(double temp1, double temp2)
{ // les deux paramètres sont en degrés
  Celsius
    System.err.println("temp1 = " + temp1
    + "\ntemp2 = " + temp2);
    return (temp1+temp2);
}
```

Workflow : GitHub (suite)



Ce qui se passe lors d'une demande de tirage :

1. Anne fait sa demande de tirage en premier (par exemple) pour sa sous-branche et la fusionne. La branche `main` contient maintenant la version d'Anne.
2. Ensuite, Bernard fait sa demande de tirage pour sa sous-branche. GitHub essaie de la fusionner avec la branche `main` (déjà modifiée par Anne)
3. ⚡ Problème : GitHub voit que le fichier `Toto.java` a été modifié différemment par Anne et Bernard, même si ce n'est pas dans les mêmes lignes
4. Résultat : Conflit de merge !

Workflow : GitHub (suite)



- Les exemples au-dessus sont volontairement simplistes mais illustrent ce type de problèmes
- D'autres cas possibles portent sur des fichiers supprimés dans certaines branches, et modifiées dans d'autres...
- Des solutions existent, mais ne résolvent pas tous les problèmes
- Git choisi de laisser faire et de juste signaler le problème quand il se pose et qu'il ne sait pas le résoudre automatiquement. Aux utilisateurs de décider quoi faire !
 - Les lignes posant problèmes seront juste signalées

Workflow : GitHub (suite)



```
double addTemp(double temp1, double temp2) {  
<<<<<<< main  
    if (temp1 <= -273.15 || temp2 <= -273.15) {  
        throw new IllegalArgumentException  
            ("sous zéro absolu !");  
    }  
  
=====  
        System.err.println("temp1 = " + temp1  
+ "\ntemp2 = " + temp2);  
>>>>>>> Bernard_verification  
        return (temp1+temp2);  
}
```

Workflow : GitHub (suite)



- Dans les deux exemples, il peut être décidé
 - de garder la version d'Anne (et oublier celle de Bernard)
 - ou de garder la version de Bernard (et oublier celle d'Anne)
 - ou d'écrire un code qui combine les deux d'une façon satisfaisante

Workflow : GitHub (suite)



Code possible combinant les deux sous-branches :

```
double addTemp(double temp1, double temp2)
{ // les deux paramètres sont en degrés Celsius
    System.err.println("temp1 = " + temp1 + "\ntemp2
= " + temp2);
    if (temp1 <= -273.15 || temp2 <= -273.15) {
        throw new IllegalArgumentException
            ("sous zéro absolu !");
    }
    return (temp1+temp2);
}
```

Workflow : GitHub (suite)



- 'pull request' ("demande de tirage") (suite) :
 - ...
 - Il est même possible que la demande de tirage soit bloquée, alors qu'elle pourrait être faite automatiquement !
 - c'est le cas si des ajouts sont très proches dans la même zone de code. GitHub ne peut prendre la décision car il ne peut déterminer seul quelle modification doit être écrite avant l'autre (même dans les cas où l'ordre n'a pas d'importance)
 - il faut alors forcer la demande de tirage en fixant un ordre dans les codes ajoutés...
 - ...

Workflow : GitHub (suite)



- 'pull request' ("demande de tirage") (suite) :
 - Conseils divers :
 - toujours essayer de garder à jour les versions locales de branches de base sur lesquelles vous allez travailler ('pull', à ne pas confondre avec le mot 'pull' dans 'pull request')
 - toujours faire un 'pull' avant de faire un 'push', juste pour limiter encore plus les risques
 - ...

Workflow : GitHub (suite)



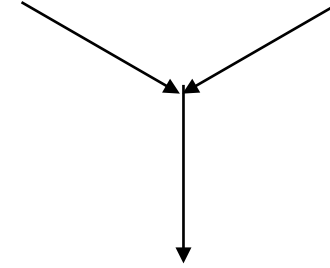
- 'pull request' ("demande de tirage") (suite) :
 - 'review' ("révision") humaine : phase de discussion, vérification et acceptation des modifications apportées, par d'autres membres du projet.
 - il s'agit d'avoir des avis sur la modification du code : acceptable ? pourquoi ?
 - ...

Workflow : GitHub (suite)



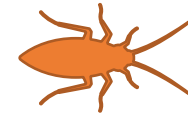
- 'pull request' ("demande de tirage") (fin) :
 - ...
 - Dans certains cas, la phase de "révision" est considérée comme tellement utile qu'elle peut être rendue obligatoire avant toute fusion
 - il est même possible de demander des vérifications au fur et à mesure de l'avancée des modifications par une 'draft pull request' ("demande de tirage brouillon")

Workflow : GitHub (suite)



- 'merge' ("fusion") : incorporation des modifications d'une branche supplémentaire, dans la branche de base de cette branche.
 - Généralement faite après une "demande de tirage" acceptée.
- Une branche qui a été fusionnée dans sa branche de base devient inutile et devrait être effacée
- Note: si X est une branche de base de la branche Y, et la branche Y est une branche de base de la branche Z, alors la fusion de Y dans X fait que X devient la branche de base de la branche Z (Y a "disparu" entre X et Z)

Workflow : GitHub (suite)



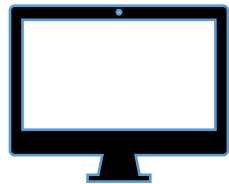
- 'issue' ("problème") : problème (bogue) "officiellement" signalé dans l'espace dédié sur GitHub à un dépôt
 - inclus aussi des demandes d'ajouts de fonctionnalités ('feature').
 - inclus aussi des retours d'expériences ('feedback').
 - zone de discussion autour de ce problème.
 - état actuel : qui s'en occupe, toujours en cours, ne sera pas traité, etc.

Workflow : GitHub (suite)



- 'clone' : copie "locale" (sur un PC ou dans une VM) d'un dépôt situé à distance (sur GitHub par exemple).
- 'push' ("poussée") : envoi des modifications d'une branche "locale" vers la version de cette branche dans un dépôt "à distance".

Les 'Codespaces' de GitHub



Workflow : GitHub (suite)



- 'Codespaces' : environnements de développement virtualisés gérés et exécutés par GitHub dans son 'Cloud'.
 - ils permettent de travailler directement dans un environnement de développement sans rien configurer (ou presque)
 - bouton "+" en haut à droite -> "new codespace" pour créer un nouveau Codespace dans la branche actuelle,
 - ou aller au plus haut niveau de la branche temporaire à modifier et tester -> bouton "<> Code" -> Codespaces -> choisir un Codespace existant

Workflow : GitHub (suite)



- 'Codespaces' (suite) :
 - ...
 - la branche choisie est clonée directement dans le Codespace
 - on peut alors travailler dans le Codespace ouvert
 - Tout Codespace considère github.com comme un dépôt distant dont une branche est clonée dans ce Codespace !
 - Il n'y a pas copie automatique des modifications faites dans le Codespace vers le dépôt GitHub => il faut demander une "poussée" ('push') avant de l'arrêter
 - ...

Workflow : GitHub (suite)



- ‘Codespaces’ (suite) :
 - ...
 - faire des ``git commit -a`` dans le Codespace pour créer un commit local à ce Codespace (pas dans le dépôt d'origine !), faire des ``git add ...`` avant si des fichiers ont été ajoutés manuellement dans le Codespace
 - faire ``git push`` pour pousser (‘push’) les commit locaux vers la branche actuelle dans GitHub !
 - ...

Workflow : GitHub (suite)



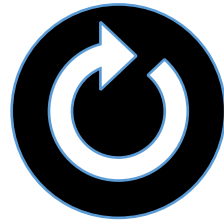
- 'Codespaces' (suite) :
 - ...
 - ne pas laisser tourner inutilement un Codespace !!!
 - compte gratuit personnel = 120 heures "cœur" / mois, avec 2 cœurs min par VM ?
 - et 15 Go /mois total
 - pour arrêter un codespace : (dans un terminal du codespace), taper ``gh codespace stop`` et choisir le codespace à arrêter
 - ...

Workflow : GitHub (suite)

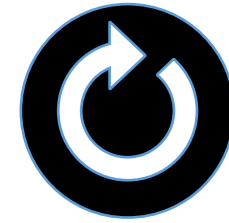


- 'Codespaces' (fin) :
 - ...
 - on peut aussi arrêter un Codespace avec le bouton bleu en bas à gauche de la fenêtre Codespace.
 - note : ils sont exécutés chez Azure (Microsoft) car Microsoft est propriétaire de GitHub.

Retour vers le Workflow GitHub

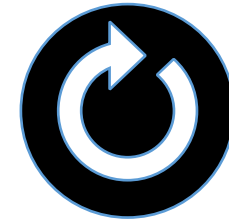


Workflow : GitHub (suite)



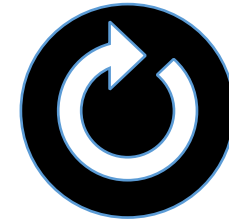
- Travail préparatoire pour le workflow GitHub :
 - commencer par préciser dans les paramètres que la branche 'main' doit être protégée par une obligation de demande de tirage avant qu'une fusion d'une branche avec cette branche main puisse avoir lieu.
 - la protection permet de s'assurer que personne ne peut « abimer » accidentellement la branche principale.
 - On peut même préciser que seul le propriétaire du dépôt est autorisé à valider la demande de tirage et faire la fusion

Workflow : GitHub (suite)



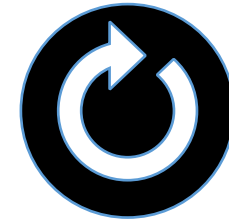
- Le GitHub flow conseille fortement ("best practice") de continuer ensuite, avant tout travail, par créer une 'issue' avec un titre (s'il n'y en a pas déjà une pour ce travail)
 - dans le dépôt, onglet « Issue »
 - ajouter un ou plusieurs labels standards si désiré
 - bug, enhancement, documentation
 - décrire le problème ou la demande
 - valable même avant même la 1ere ligne de code écrite

Workflow : GitHub (suite)



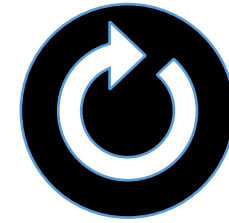
- le GitHub flow demande ensuite de commencer à traiter une demande en créant une nouvelle branche à partir de la branche 'main' (ce qui correspond au traitement de cette demande – fonctionnalité, bogue, documentation, etc.)
 - dans le dépôt, onglet "Code" -> menu déroulant pour choisir la branche existante comme branche de base de la nouvelle branche -> nommer la nouvelle branche (généralement un verbe d'action + objet), etc.
 - la branche de base habituelle est la branche 'main'
 - mais on peut choisir une autre branche

Workflow : GitHub (suite)



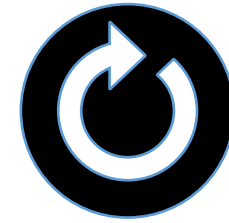
- Vérifier qu'on travaille bien dans la nouvelle branche.
- ... (travail) ...
- si on travaille localement, en dehors du dépôt (dans un Codespace par exemple, ou sur son PC), on fait un push vers GitHub ('pushing the commits').
- une fois satisfait du résultat, on fait une « demande de tirage »
 - inclure un résumé des modifications et du problème qu'elles résolvent.

Workflow : GitHub (suite)



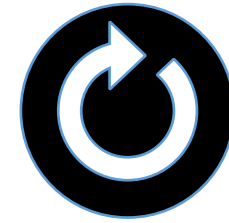
- on procède alors aux tests automatiques
 - Ils sont un des points importants des méthodologies DevOps et CI/CD !
- si tests satisfaisants, on fait la demande de « révision » ('review')
- si la relecture est acceptée, on procède à la « fusion »
- en cas de problème dans l'une des étapes, on revient en arrière pour corriger le problème

Workflow : GitHub (suite)



- Quand la fusion a été effectuée :
 - on supprime généralement la branche qui a été fusionnée, pour éviter de travailler par erreur plus tard sur cette branche devenue fermée.
 - L'historique permettra de les retrouver plus tard si nécessaire.

Workflow : GitHub (fin)



- Mais dans certains cas (liés par exemple à l'existence d'une application de « ticket » de problèmes), il peut être intéressant de ne pas supprimer les branches qui ont été fusionnées à leurs branches de base
 - par exemple pour garder la liaison entre un « ticket » et la branche qui corrige le problème,
 - mais l'espace des branches grossit et peut devenir très touffu...